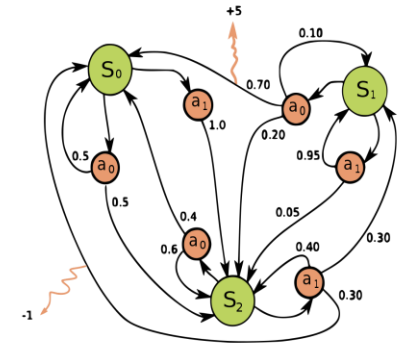


COSC2129/1476

Artificial Intelligence

Markov Decision Processes

Slides from Prof. Sebastian Sardiña



Artificial Intelligence - Road map

8 Markov Decision Processes

9 Reinforcement Learning

10 Bayesian Reasoning

11 Review + AI Philosophy + Course recap



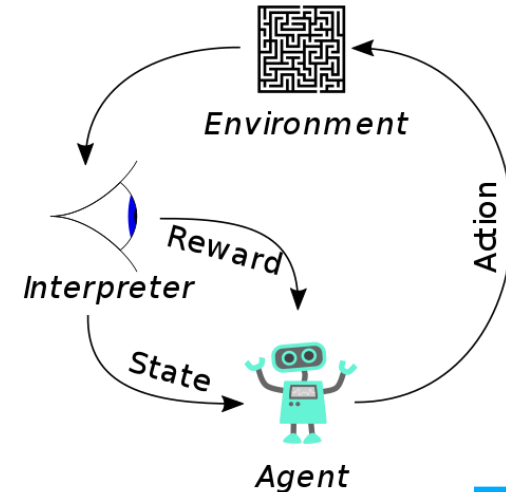
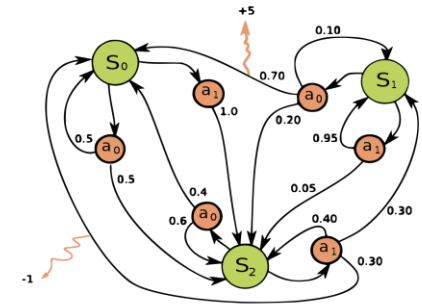
Decision making under uncertainty: Markov Decision Processes

Book AIMA, Chapter 15.1-15.3, 16.1-16.2

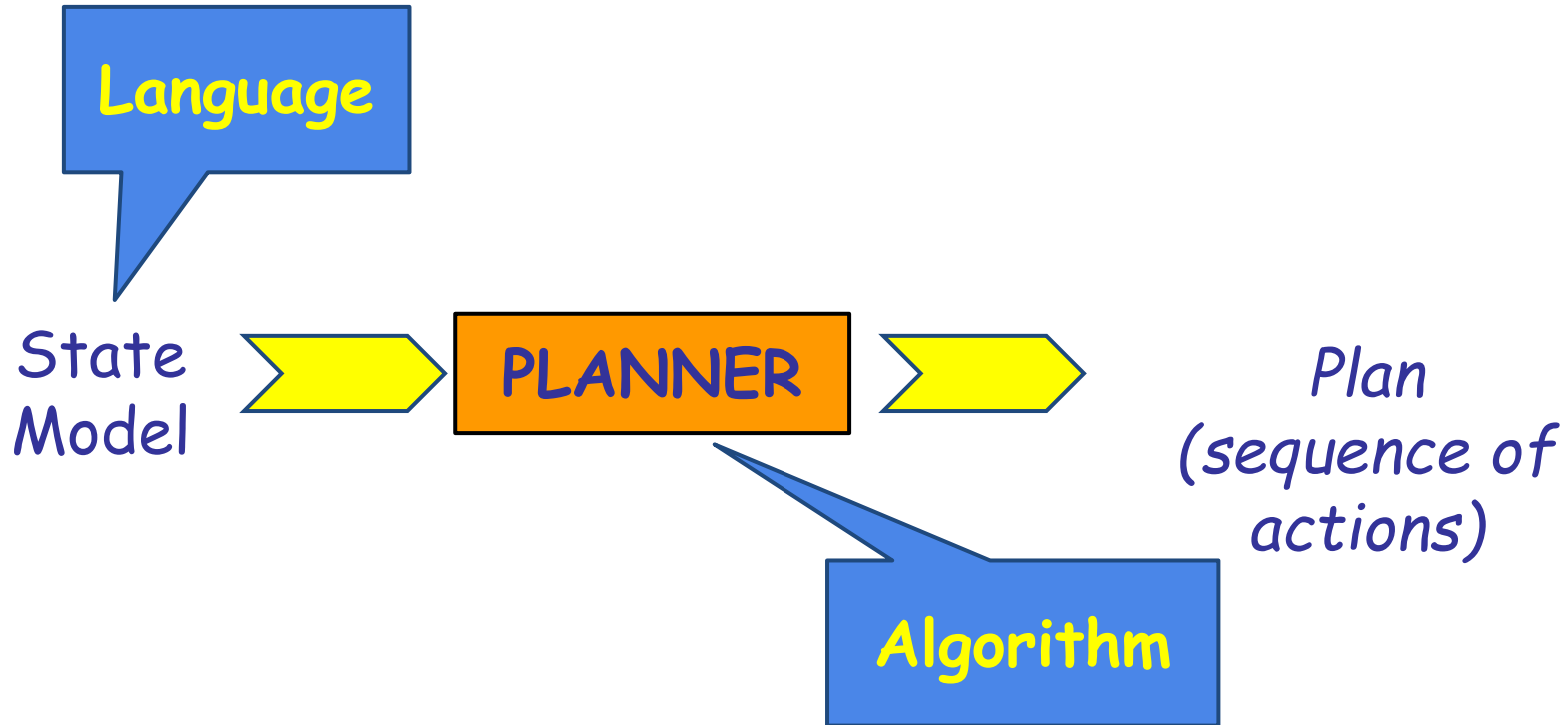
15 Making Simple Decisions	518
15.1 Combining Beliefs and Desires under Uncertainty	518
15.2 The Basis of Utility Theory	519
15.3 Utility Functions	522
15.4 Multiattribute Utility Functions	530
15.5 Decision Networks	534
15.6 The Value of Information	537
15.7 Unknown Preferences	543
Summary	547
Bibliographical and Historical Notes	547
16 Making Complex Decisions	552
16.1 Sequential Decision Problems	552
16.2 Algorithms for MDPs	562
16.3 Bandit Problems	571
16.4 Partially Observable MDPs	578
16.5 Algorithms for Solving POMDPs	580
Summary	585
Bibliographical and Historical Notes	586

Outline

- Sequential decision problems
- Markov decision processes
 - Definition
 - Policy
 - Utilities of Sequences
- Value Iteration
 - Utilities of States
 - Bellman equations
 - Iterative algorithm
- Policy Evaluation & Policy Iteration



Week 6: Classical Planning



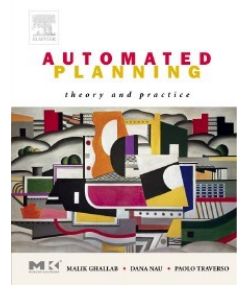
Recap Previous Weeks

Automated planning:

Goals, planning, logic representation, search, ...

E.g: *To be on the flight ontime.*

Traffic? Accidents? Security checks?
Flight delays?



Properties/Assumptions:

World is fully observable.

Deterministic actions.

Static environment.

Goal is "all or nothing".

But:

- Knowledge is *incomplete*.
- Actions are *fallible & non-deterministic*.
- Environment is *dynamic*.
- Agent uses *preferences*.

Probability

Probability theory very well established.

Use it to assign degrees of beliefs and likelihoods to (uncertain) events.

E.g. $\Pr(\text{coin} = \text{head}) = \frac{1}{2}$



We have seen this 2
weeks ago; let's
use them!

Before that...

Did you prove Bayes Rule?

LIKELIHOOD

The probability of "B" being True, given "A" is True

PRIOR

The probability "A" being True. This is the knowledge.

$$P(A|B) = \frac{P(B|A).P(A)}{P(B)}$$

POSTERIOR

The probability of "A" being True, given "B" is True

MARGINALIZATION

The probability "B" being True.

Making decisions? Utility Theory

Deterministic logical agent has a goal and executes (any) plan that achieves the goals.

In an **uncertain** world, this is no longer the case.

Flight A90: 95% making to airport on time.

Flight A120: 98% making it on time.

Flight A1440: 100% making it on time.

Relax goals: consider agent **preferences** for different outcomes.

Utility theory = associate degree of usefulness (utility) to each outcome, with outcomes of higher utility preferred.

Simple vs Complex Sequential Decisions

Simple decisions: utility of plan based on **one** action/decision:

Whether I take a taxi or train to airport?

Where to go for a date?

Complex Sequential decisions: utility of outcomes are dependent on a **sequence of actions/decisions:**

Chess.

Financial planning.

Path navigation.

Deployment of mining trucks in an area.

Overview of next 2 weeks

1. Sequential Complex Decision Making: MDPs

Environment known, but uncertainty in outcomes of actions/decisions.

Utility of plan dependent on sequence of actions.

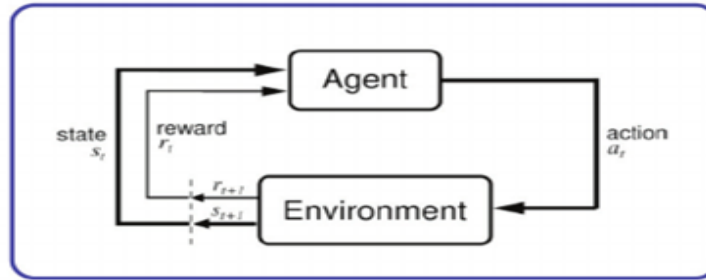
2. Reinforcement Learning

Environment unknown (transitions, rewards).

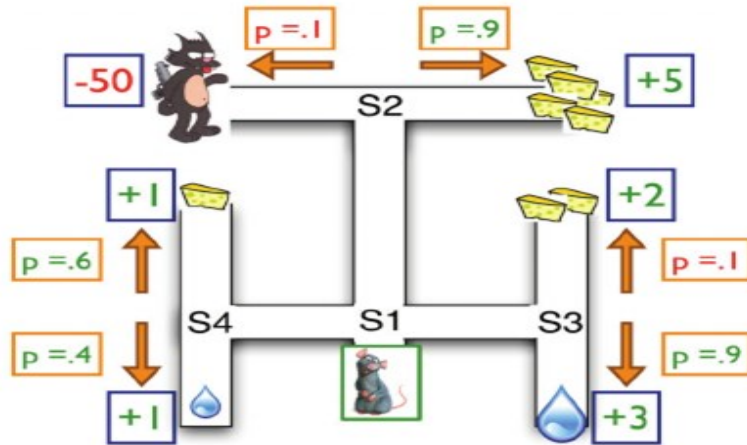
But *feedback available!*

MDP: Markov Decision Processes

A



B



C

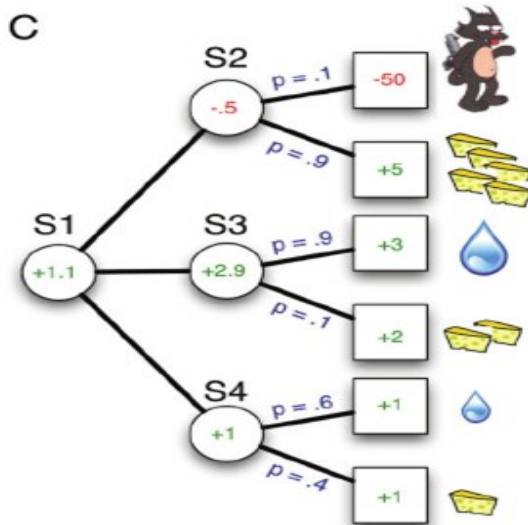


Image from: [A Primer on Reinforcement Learning in the Brain: Psychological, Computational, and Neural Perspectives](#)

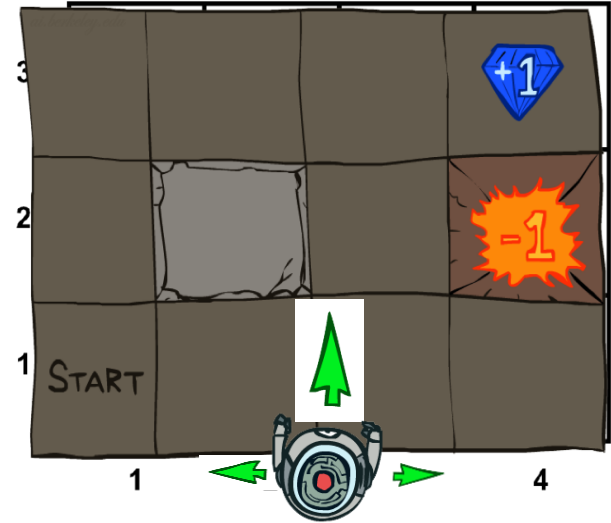
Outline of MDP Content

1. GridWorld Running Example.
2. Definition of Markov Decision Processes.
3. Policies.
4. Utilities of Sequences.
5. Value Iteration.
6. Policy Evaluation & Policy Iteration.



Illustrative Example: GridWorld

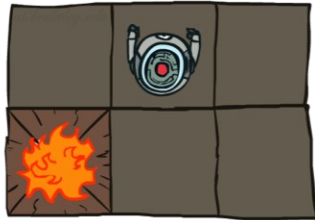
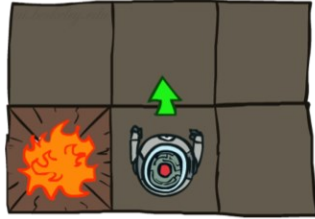
- A maze-like world
 - Agent lives in a grid
 - Walls block agent's path
- **Movement actions are noisy** - they do not always go as planned.
 - When go north, 80% go north, but 10% of time go west, 10% east.
- **Agent receives rewards** each time step
 - Small "living" reward each step (can be negative)
 - Big rewards comes at the end (good or bad)



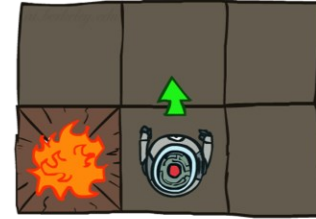
Question: How to find the best sequence of actions or plan an agent should take in GridWorld?

GridWorld Actions

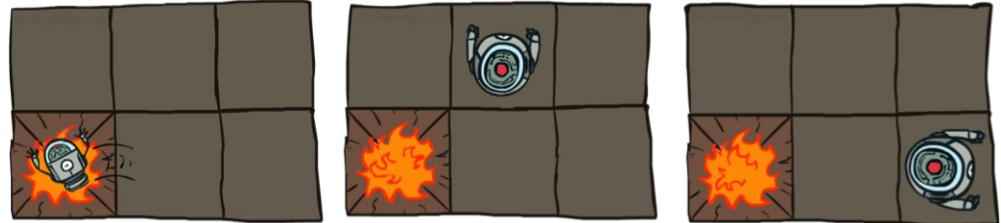
Deterministic Grid World



Stochastic Grid World

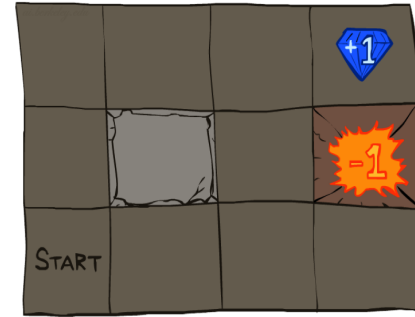


?



Characteristics of GridWorld

- Environment is **fully observable**.
 - Agent knows where it is at any time.
- But **actions are uncertain** (movement).
 - Each grid location is a state that the agent can be in.
- Has terminating grid locations/states.
- **Sequential decision** problem.
 - Each grid location/state has a **reward**.
 - Final utility depends on a sequence of states agent traversed.

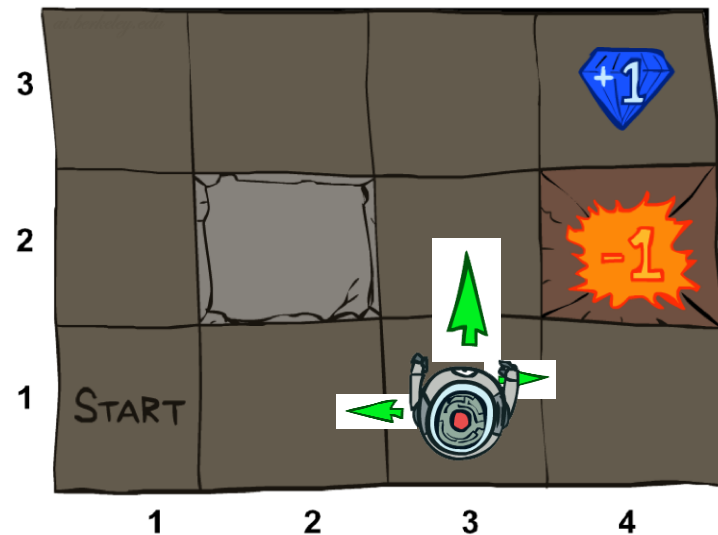


Question: How to find the best sequence of actions an agent should take in GridWorld?

Markov Decision Processes (MDP)

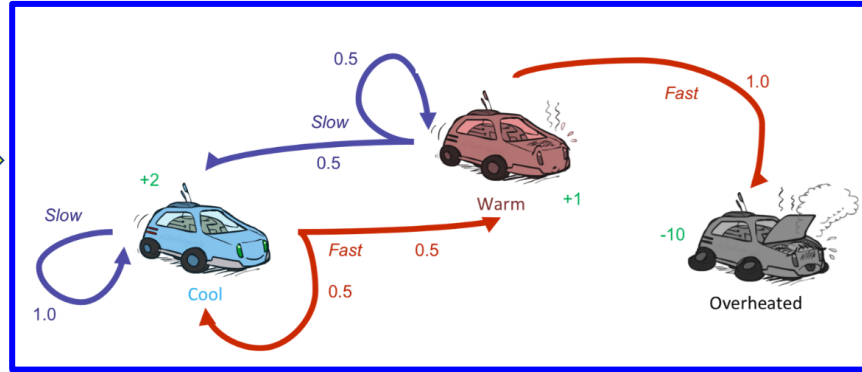
An MDP is a tuple $M = (S, A, T, R, s_0, F)$:

1. A set of states $s \in S$:
locations, clean, fuel, broken, ..
2. A set of actions $a \in A$:
move to, suck, pick, drop, ...
3. A transition function T ("the model"):
 $T(s, a, s')$: Probability that a
in s leads to s' , i.e., $P(s' | s, a)$
pick action succeeds 90%
4. A reward function $R(s)$: value of state s .
clean living room gives +10
5. A start state $s_0 \in S$.
robot initial location, etc..
6. Possibly terminal states $F \subseteq S$

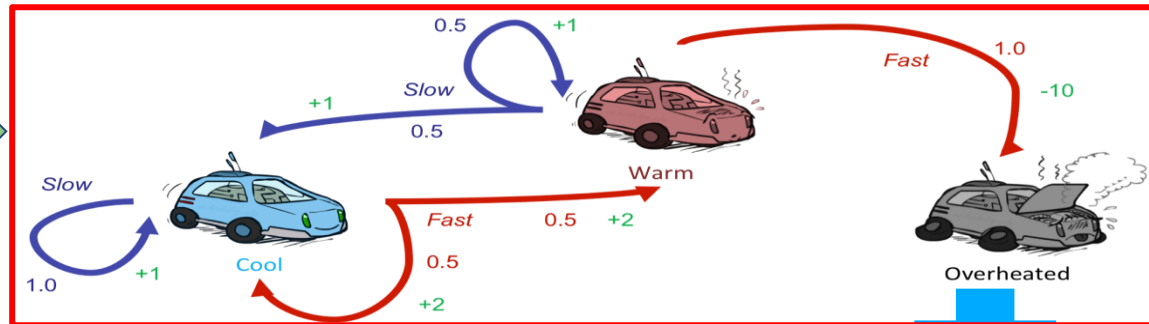


Rewards on states or transitions?

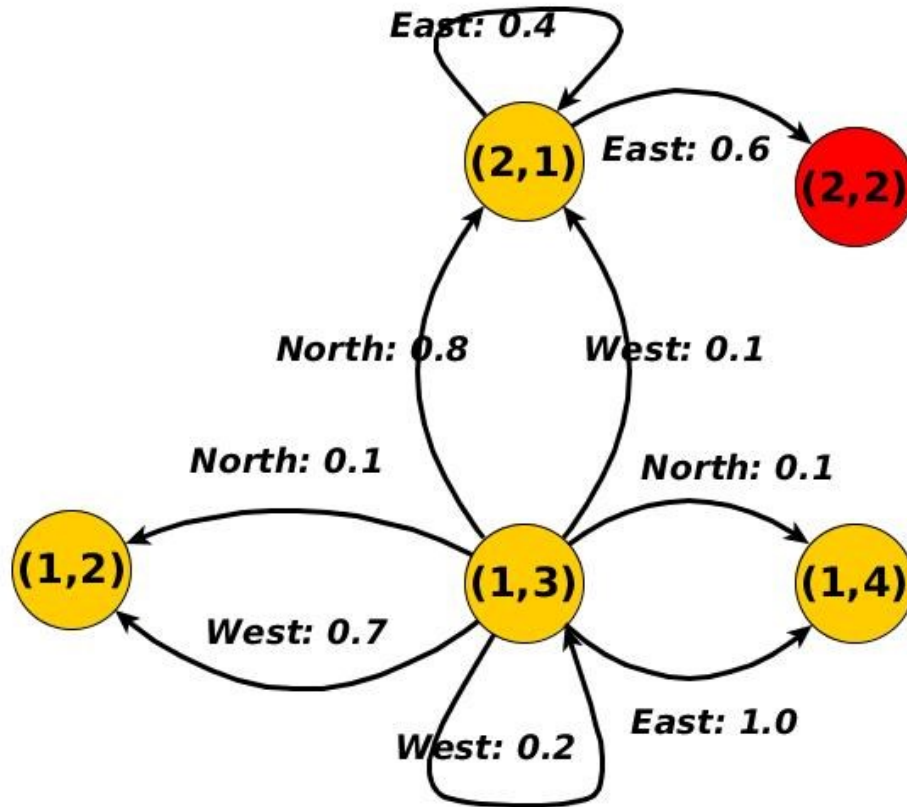
$R(s)$: rewards on states
(book R&N & here).



$R(s, a, s')$: rewards on transitions ("standard")



Transition System / Model



Note: This is only a portion of the whole transition model for GridWorld. States only show location, more features may be in a state

What is Markovian in MDP?

- “Markov” generally means that given the present state, the future and the past are independent.
- For MDPs, “Markov” means **action outcomes depend only on the current state.**
 - E.g., in GridWorld, the **outcomes of the agents movement actions** (go north, can go north, west or east) only depends on its **current grid cell**.

Outline of MDP Content

1. GridWorld Running Example
2. Definition of Markov Decision Processes
3. Policies
4. Utilities of Sequences
5. Value Iteration
6. Policy Evaluation & Policy Iteration



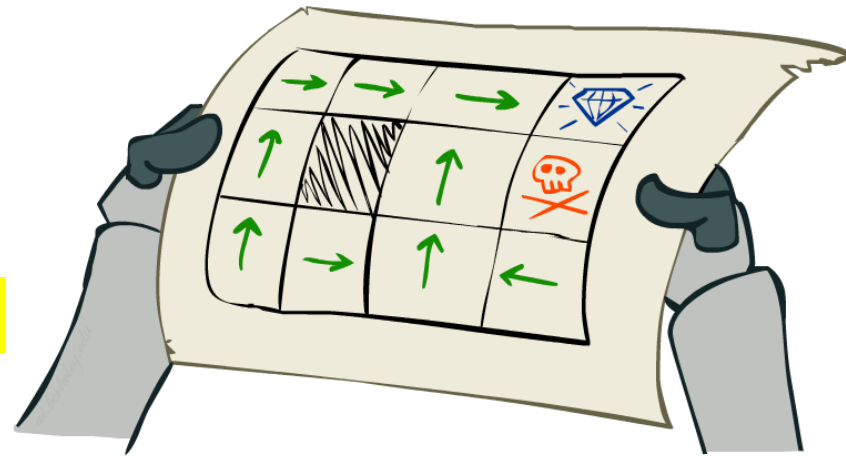
MDP Solution: A “Good” Policy

In deterministic single agent problems, an (optimal) sequence of actions from start to goal is enough.

In MDP, we want an optimal policy π^* :

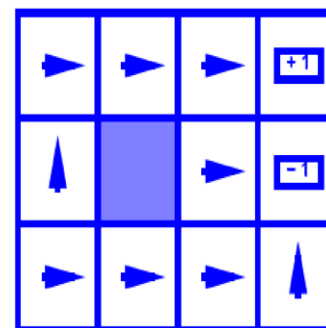
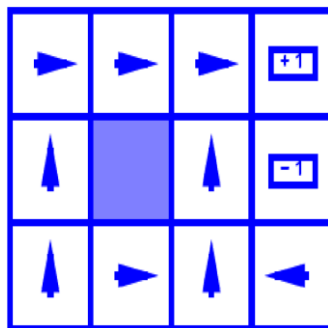
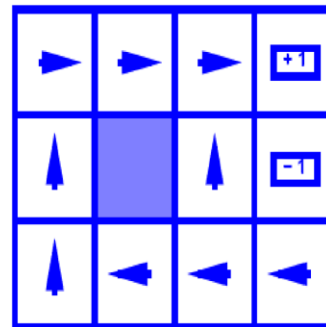
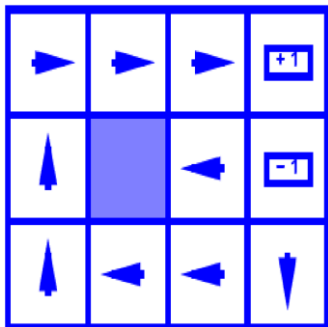
$S \rightarrow A$

- A **policy** specifies what an agent should do (action) for each state.
- An optimal policy is one that maximises “expected utility” (more on this soon)



Optimal policy when
 $R(s) = -0.03$ for all non-terminals s

Four different policies



Outline of MDP Content

1. GridWorld Running Example
2. Definition of Markov Decision Processes
3. Policies
- 4. Utilities of Sequences**
5. Value Iteration
6. Policy Evaluation & Policy Iteration

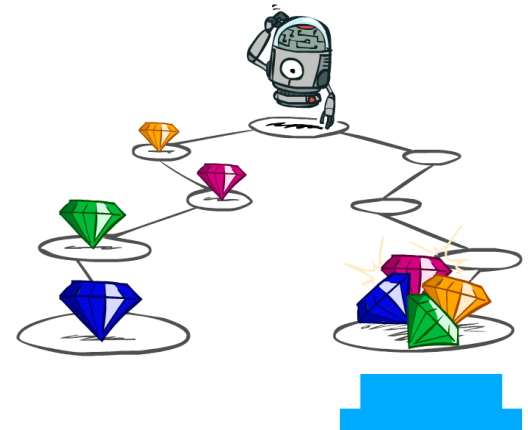


Utilities of Sequences

Recall basic rewards represents agents "preferences".

How to measure preferences/rewards/utilities over a sequence of states?

- ❑ Less now, but better after:
 - ❑ $[-10, 0, 1000]$ or $[10, 1, 1]$
- ❑ Now or Later?
 - ❑ $[10, 0, 0]$ or $[0, 0, 10]$



Discounting

It's reasonable to maximise the **sum** of rewards.

It's also reasonable to prefer rewards **now** to later.

One solution: *values of rewards decay exponentially.*



1

Worth
Now



γ

Worth Next
Step



γ^2

Worth In Two
Steps



How does discounting work?



Given a sequence of states $[s_1, s_2, s_3, \dots, s_n]$ and discount factor $\gamma \in (0,1]$, the utility of starting at state s_1 can be defined as:

$$U(s_1) = R(s_1) + \gamma R(s_2) + \gamma^2 R(s_3) + \dots + \gamma^n R(s_n)$$



Stationary Preferences Assumption

Theorem: if we assume stationary preferences:

$$\begin{aligned}
 [a_1, a_2, \dots] &\succ [b_1, b_2, \dots] \\
 &\Updownarrow \\
 [r, a_1, a_2, \dots] &\succ [r, b_1, b_2, \dots]
 \end{aligned}$$

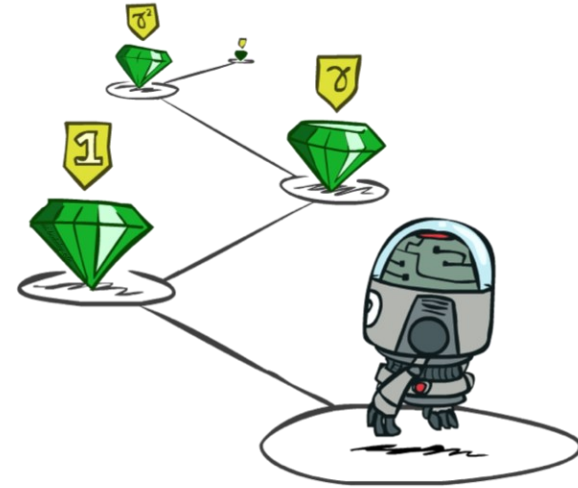
Then: there are only two ways to define utilities:

Additive utility

$$U([r_0, r_1, r_2, \dots]) = r_0 + r_1 + r_2 + \dots$$

Discounted utility

$$U([r_0, r_1, r_2, \dots]) = r_0 + \gamma r_1 + \gamma^2 r_2 \dots$$



Infinite Horizons



Problem: What if the game lasts forever (no termination state, or agent never reaches one)? Infinite utilities for additive?

Solutions:

- **Proper policies:** policies that guarantee agent always reach a terminal state. ****Not always possible****.
 - e.g., classical planning or control.
- **Finite horizon:** terminate game after a fixed number of steps.
 - But gives non-stationary policies (they have time dependencies)
- **Discounted rewards:** - utility is finite ($0 < \gamma < 1$)



Recap: MDPs

- Markov decision processes (MDP):
 - Set of states S
 - Start state s_0
 - Set of actions A
 - Transitions $T(s,a,s')$
 - Rewards $R(s)$
- MDP concepts so far:
 - Policy = choice of action for each state
 - Utility = sum of discounted rewards

Outline of MDP Content

1. GridWorld Running Example
2. Definition of Markov Decision Processes
3. Policies
4. Utilities of Sequences
- 5. Value Iteration**
6. Policy Evaluation & Policy Iteration



Utility of states & Value Iteration

Given a MDP: **how to find the optimal policy for it?**

Optimal policy indicates for each state, what is the best action to perform?

In order to determine what is the best action, need to calculate the utility of each state. Recursive? How to calculate this?

Value Iteration



Utility of states - Deterministic Actions

Assume **deterministic actions**, i.e, that is:

- For all s, a, s' : $T(s, a, s') = 1$ or $T(s, a, s') = 0$.
- Then a policy from s_1 gives a sequence $s_1, s_2, s_3, \dots$

What is the utility of state s_1 when following policy π ?

$$U^\pi(s_1) = R(s_1) + \gamma R(s_2) + \gamma^2 R(s_3) + \dots = \sum_{t \geq 0} \gamma^t R(s_t)$$

s_{i+1} = the state after doing action $\pi(s_i)$ in state s_i

that is, the state such that $T(s_i, a, s_{i+1}) = 1$

Utility of states - Deterministic Actions



$$U^\pi(s_1) = R(s_1) + \gamma R(s_2) + \gamma^2 R(s_3) + \gamma^3 R(s_4) + \dots$$

$$U^\pi(s_1) = R(s_1) + \gamma [R(s_2) + \gamma R(s_3) + \gamma^2 R(s_4) + \dots]$$

$$U^\pi(s_1) = R(s_1) + \gamma U^\pi(s_2)$$

This is based on using a particular policy π .

But π is what we are trying to find!

The utility of a state $U(s)$ is when **we choose**

the optimal

$$U(s) = R(s) + \gamma \max_a \{T(s, a, s') U(s')\}$$

The maximum value
across all possible
actions.



Utilities of States

Now let's go back to MDPs, where **actions are uncertain**.

$$\square T(s, a, s') = P(s' \mid s, a)$$

deterministic domain

$$U(s) = R(s) + \gamma \max_a \{T(s, a, s')U(s')\}$$

Bellman Equation



$$U(s) = R(s) + \gamma \max_a \{\sum_{s' \in S} T(s, a, s')U(s')\}$$

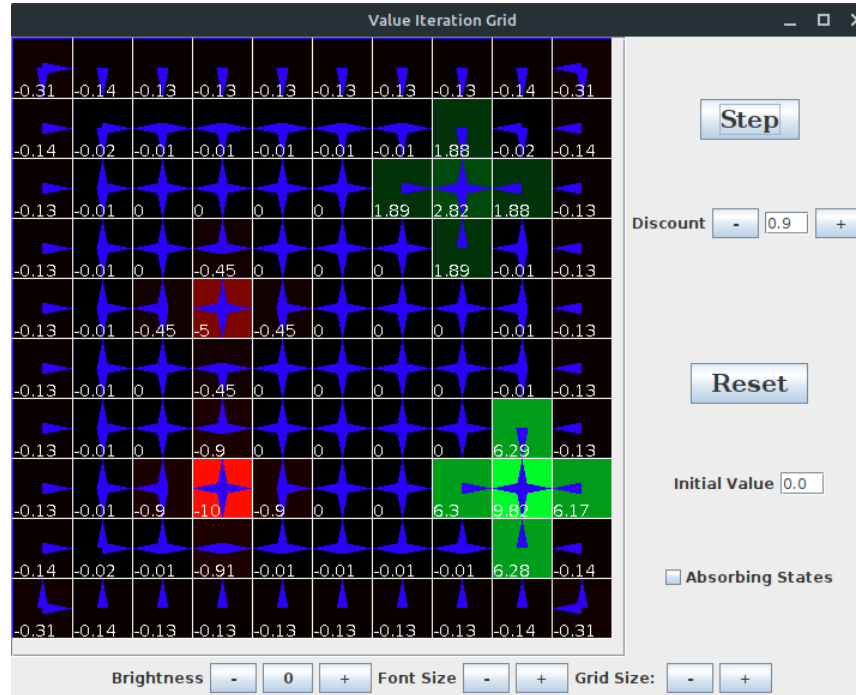
Gridworld Utilities

3	0.812	0.868	0.918	+ 1
2	0.762		0.660	-1
1	0.705	0.655	0.611	0.388
	1	2	3	4

Utilities of state for $\gamma=1$ and $R(s) = 0.04$ for non-terminal states.

Value Iteration Gridworld App

Run VI step by step using this tool I refactored as a stand-alone Java application:



Get it at <https://github.com/COSC1127/ai-tools>

Maximum Expected Utility Principle

Suppose that we have the optimal utility at every state, that is we have $U(s)$. What is the optimal policy?

The **Maximum Expected Utility (MEU) Principle** says to find the optimal action, we chose the action that **maximises** the **expected** utility of the **subsequent** states:

$$\pi^*(s) = \operatorname{argmax}_a \underbrace{\sum_{s'} T(s, a, s') U(s')}_{\text{expected utility over } s'}$$

Value Iteration

For each state s we have its Bellman equation (so we have $|S|$ equations):

$$U(s) = R(s) + \gamma \max_a \{ \sum_{s' \in S} T(s, a, s') U(s_2) \}$$

But we cannot solve as simultaneous equations due to the max operator, but can **solve iteratively**:

- $U_0(s) = 0$, for all states $s \in S$
- $U_{i+1}(s) = R(s) + \gamma \max_a \{ \sum_{s' \in S} T(s, a, s') U_i(s_2) \}$

Guaranteed to converge!

Value Iteration

Value Iteration Bellman Dynamic Programming
 repeats $v(s) \leftarrow 0$
 $\forall s \in S$
 $\delta \leftarrow 0$
 for $s \in S$
 $v'(s) \leftarrow r(s) + \gamma \max_a \sum_{s' \in S} T(s, a, s') v(s')$
 if $|v'(s) - v(s)| > \delta$
 Then $\delta \leftarrow |v'(s) - v(s)|$
 until $\delta < \epsilon(1-\gamma)/\gamma$
 return v
 Error utility for best action

Function valueIteration(MDP, ϵ , γ)

$U(s) = 0$, for all $s \in S$

Repeat

$U' = U$ // save existing U into U'

$d = 0$ // largest difference between U and U'

For each state $s \in S$ do: // get new U

$$U(s) = R(s) + \gamma \max_a \{ \sum_{s' \in S} T(s, a, s') U'(s') \}$$

$$d = \max \{ d, |U(s) - U'(s)| \} \quad // \text{store max}$$

error

Until d is very small

Extracting Optimal Policy

After value iteration, we know the utility of each state for an optimal policy. We have $U(s)$ function, for all s .

To **extract** the optimal action for each state (i.e., the policy), we use:

$$\pi^*(s) = \operatorname{argmax}_a \underbrace{\sum_{s'} T(s, a, s') U(s')}_{\text{expected utility over } s'}$$

The **Maximum Expected Utility (MEU) Principle** says to find the optimal action, we chose the action that **maximises** the **expected** utility of the **subsequent** states:

Outline of MDP Content

1. GridWorld Running Example
2. Definition of Markov Decision Processes
3. Policies
4. Utilities of Sequences
5. Value Iteration
6. Policy Evaluation & Policy Iteration
(just know it conceptually; no exercises)



Policy Evaluation

In the case where the policy is **fixed** (e.g, always move right if possible), policy evaluation is to calculation the utility of each state for the fixed policy.

Given policy π :

$$U_{k+1}^{\pi}(s) = R(s) + \gamma \sum_{s' \in S} T(s, \pi(s), s') U_k^{\pi}(s')$$

Policy Iteration

Value Iteration can be slow to converge and finding best actions can be costly.

Alternative: Policy Iteration:

1. Initialise with some policy
2. Given current policy, evaluate it (calculate U)
3. Calculate new policy using MEU and U
4. Repeat until no changes in policy.

Value Iteration vs Policy Iteration

Both value iteration and policy iteration compute the optimal values

Value iteration

- Every iteration updates utility values and policies implicitly

- We don't track the policy but taking the max over all actions implicitly computes it

In policy iteration

- Utilities are updated over a fixed policy (fast)

- After utilities calculated, find optimal policy (similar to value iteration)

Complexity Comparison

Policy iteration

Generally less iterations than value iteration:

Policy evaluation can be solved in $O(|S|^3)$

MEU can be costly $O(|A||S|^2)$ per run

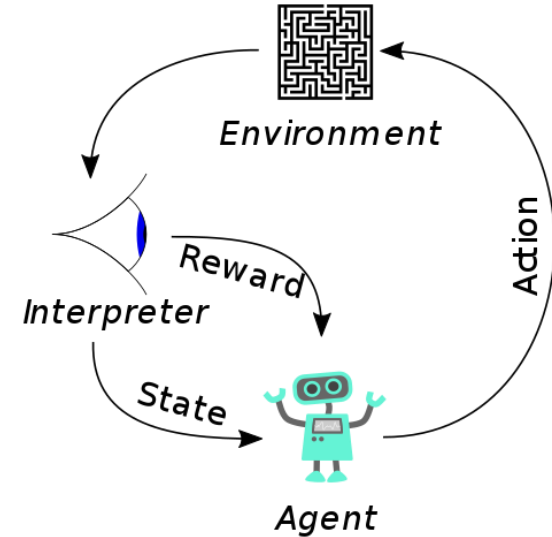
Value iteration:

$O(|A||S|^2)$ per iteration

Can have many iterations to convergence

Conclusion

- Sequential decision problems.
- Markov decision processes:
 - Definition
 - Policy: map $S \rightarrow A$
 - Utilities of Sequences
- Value Iteration:
 - Utilities of States
 - Bellman equations
 - Iterative algorithm
- Policy Evaluation & Policy Iteration:



Next lecture: Reinforcement learning!

Whiteboard used...

Function valueIteration(MDP, ϵ , γ)

$U(s) = 0$, for all $s \in S$

Repeat

$U' = U$ // save existing U into U'

$d = 0$ // largest difference between U and U'

For each state $s \in S$ do: // get new U

$$U(s) = R(s) + \gamma \max_a \left\{ \sum_{s' \in S} T(s, a, s') U'(s') \right\}$$

$$d = \max \{d, |U(s) - U'(s)|\}$$

Until d is very small

S	U_0	U_1	U_2	U_3
s_1	0	-5	0	0
s_2	0	5	54	0
s_3	0	10	0	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_n	0	100	120	0

Handwritten notes on the table: A red box highlights the row for s_2 . A green line connects the value 54 in $U_2(s_2)$ to the calculation below. A red line connects the value 100 in $U_1(s_n)$ to the calculation below. A blue line connects the value 5 in $U_1(s_2)$ to the calculation below. A red line connects the value 54 in $U_2(s_2)$ to the calculation below.

$$U_2(s_2) = R(s_2) + \gamma \max_a \left\{ \sum_{s'} T(s, a, s') U_1(s') \right\}$$

$$5 + \gamma (.1 \times -5 + .9 \times 100) = 54$$

$$U_1(s_2) = R(s_2) + \gamma \max_a \left\{ \sum_{s'} T(s, a, s') U_0(s') \right\}$$

Questions?

